µCRM

Documentation technique – MicroCRM

Industrialisation de la chaîne CI/CD de l'application full-stack

Auteur : Emmanuel Hesry, développeur full-stack
Destinataire : Maria, CTO d'Orion
Option : Option B
Version : 1.0.0
Date : 10/11/2026
Statut : version finale

Sommaire

1. Introduction....2
  1.1 Contexte du projet....2
  1.2 Objectifs de l'industrialisation....2
  1.3 Technologies principales....2
  1.4 Présentation rapide du pipeline CI/CD....3
2. Étapes de mise en œuvre du pipeline CI/CD....3
  2.1 Structure du pipeline....3
  2.2 Scripts d'automatisation....4
  2.3 Reproductibilité....5
3. Plan de conteneurisation et de déploiement....5
  3.1 Dockerfiles....5
  3.2 docker-compose.yml....7
  3.3 Déploiement....7
4. Plan de testing périodique....9
  4.1 Types de tests automatisés....9
  4.2 Fréquence d'exécution....10
  4.3 Objectifs des tests....10
5. Plan de sécurité....10
  5.1 Résultats SonarQube....10
  5.2 Analyse des risques....12
  5.3 Plan d'action et remédiation....14
6. Monitoring, métriques et KPI....15
  6.1 Mise en place du monitoring....15
  6.2 Métriques DORA....17
  6.3 KPI personnalisés....18
  6.4 Analyse synthétique....18
7. Plan de sauvegarde des données....19
  7.1 Ce qui doit être sauvegardé....19
  7.2 Procédure de sauvegarde....19
  7.3 Procédure de restauration....20
8. Plan de mise à jour....21
  8.1 Mise à jour de l'application....21

8.2 Mise à jour du pipeline CI/CD....22
8.3 Fréquence et bonnes pratiques....22
9. Conclusion....23
9.1 Améliorations apportées....23
9.2 Gains observés....23
9.3 Recommandations pour les itérations suivantes....23
10. Annexes....24
10.1 Fichiers de la chaîne CI/CD....24
10.2 Commandes utiles....24
10.3 Captures....25

1. Introduction

1.1 Contexte du projet

Orion développe MicroCRM, une application simplifiée de gestion de la relation client utilisée par ses départements technique et commercial. Elle se compose d'une API Spring Boot et d'une interface Angular.

Jusqu'à présent, l'application est déployée à la main, ce qui provoque des retards et des erreurs. Maria, CTO d'Orion, demande d'industrialiser sa chaîne de livraison avec GitHub Actions, Docker Compose et SonarQube Cloud.

1.2 Objectifs de l'industrialisation

- **Automatiser** la construction, les tests, la conteneurisation et la publication de l'application à chaque modification.
- **Contrôler la qualité et la sécurité** du code à chaque étape, avec SonarQube Cloud et le scan des images.
- **Formaliser** les plans de testing, de sécurité, de sauvegarde et de mise à jour.
- **Mesurer** le fonctionnement du pipeline et de l'application (logs, métriques DORA, KPI) pour l'améliorer.

1.3 Technologies principales

Domaine	Technologies	Remarque
Back-end	Java 21, Spring Boot 3.2.5, Spring Data REST, HSQLDB en mémoire, JUnit 5	API REST exposée sur le port 8080 (`/persons`, `/organizations`)
Front-end	Angular 17.3, npm, Karma et Jasmine	Application monopage
Build back	Gradle 8.7 (wrapper du dépôt)	Même version de Gradle sur tous les postes et en CI
Conteneurs	Docker (Dockerfile multi-stage), Docker Compose, Caddy	Caddy sert le front
CI/CD	GitHub Actions, SonarQube Cloud, GitHub Container Registry, Dependabot	
Monitoring	Elasticsearch, Logstash, Kibana	Logs de l'application, sur le poste de développement

Tableau 1 : Technologies du projet

Le README annonce « OpenJDK 17 ou plus » : en réalité, Gradle 8.7 ne s'exécute pas sous Java 25. Le pipeline utilise donc **Java 21**, la version d'exécution de l'image d'origine.

1.4 Présentation rapide du pipeline CI/CD

À chaque push et à chaque pull request, le pipeline construit et teste le back et le front en parallèle, puis SonarQube Cloud analyse le code ; il construit et scanne ensuite les images Docker. Sur un push vers `main`, il les publie en plus sur GitHub Container Registry et vérifie qu'elles démarrent. Il s'exécute aussi chaque nuit et à la demande, sans publier.

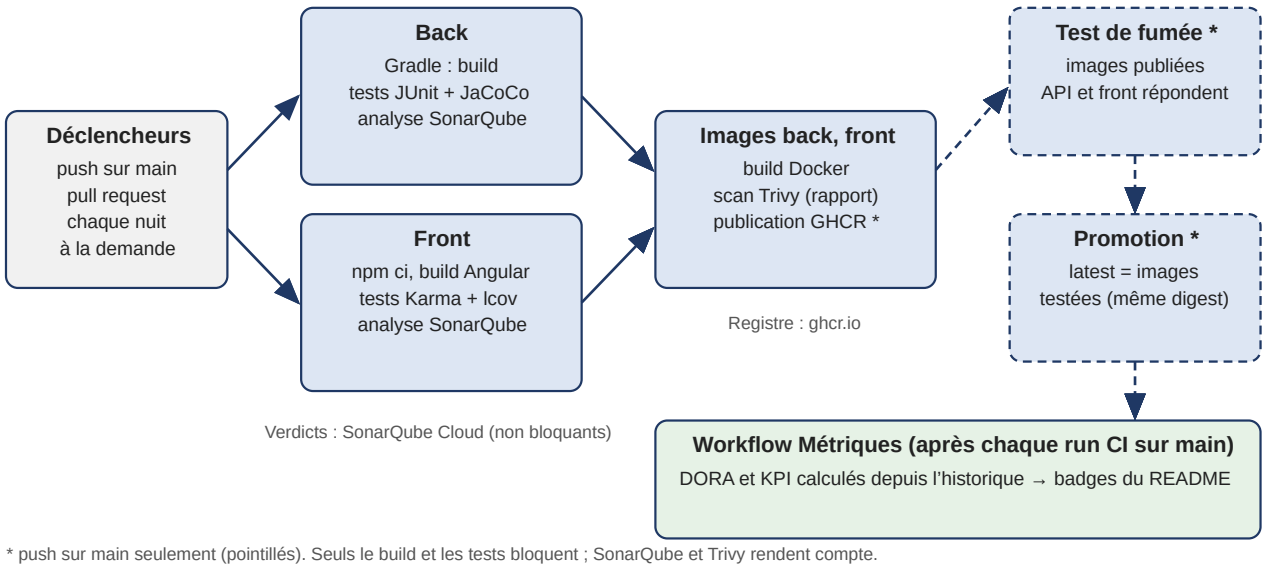


Figure 1 : Pipeline CI/CD de MicroCRM

Le détail des jobs, des étapes et de leurs outils est au § 2.1.

2. Étapes de mise en œuvre du pipeline CI/CD

2.1 Structure du pipeline

Le pipeline est défini dans un seul fichier, `.github/workflows/ci.yml`. Ses déclencheurs appliquent le plan de testing périodique (§ 4.2) : pull request vers `main`, push sur `main`, chaque nuit à 02 h 23 UTC et à la demande. L'heure de nuit est indicative : GitHub peut retarder un run planifié quand sa plateforme est chargée.

Il comporte deux jobs **indépendants, exécutés en parallèle**, chacun sur sa propre machine : le back et le front n'ont aucune dépendance de build entre eux, et chacun correspond à un projet SonarQube distinct. Dans chaque job, l'ordre est le même : on installe, on teste, on construit, puis on analyse. Un échec du build ou des tests arrête le job et bloque la fusion : une règle de protection de `main` impose une pull request et exige que les jobs Back, Front, Image back et Image front aient réussi. L'analyse SonarQube et le scan des images, eux, rendent compte sans bloquer : ils mesurent la dette du projet existant, recensée et planifiée au § 5.2, sans arrêter la livraison.

Une fois les deux jobs réussis, le job **images** construit et scanne les deux images en parallèle, à chaque run. Sur un push vers `main` seulement, il les publie, le job **test de fumée** démarre les images publiées, puis le job **promotion** fait pointer `latest` sur ces images si le test a réussi (§ 3.3).

Job back	Job front	Pourquoi cet ordre
Récupération du code	Récupération du code	Historique complet : SonarQube s'en sert pour distinguer le nouveau code

Job back	Job front	Pourquoi cet ordre
JDK Temurin 21, cache Gradle	Node.js 22, cache npm	Versions fixes ; le cache évite de retélécharger les dépendances
Build, tests et couverture JaCoCo	`npm ci`, tests Karma et couverture lcov, build de production	Les tests produisent la couverture que l'analyse lit
Analyse SonarQube, attente de la quality gate	Analyse SonarQube, attente de la quality gate	Le verdict de la quality gate est affiché, sans faire échouer le job
Rapports de tests et de couverture	Rapport de couverture	Conservés 14 jours, même en cas d'échec, pour diagnostiquer

Tableau 2 : Étapes des jobs

Action	Rôle	Pourquoi celle-ci
`actions/checkout` v7.0.1	Récupérer le code	Action officielle GitHub
`actions/setup-java` v6.1.0	Installer le JDK, cacher Gradle	Officielle ; le cache intégré évite une action de plus
`actions/setup-node` v7.1.0	Installer Node.js, cacher npm	Officielle ; même raison
`actions/upload-artifact` v7.0.2	Conserver les rapports	Officielle
`SonarSource/sonarqube-scan-action` v8.4.0	Analyser le front	Officielle de l'éditeur ; le back utilise le plugin Gradle, recommandé pour Java

Tableau 3 : Actions GitHub utilisées

Seules des actions officielles sont utilisées, chacune fixée par SHA de commit (§ 5.2). Une nouvelle version n'est adoptée qu'après un délai de 7 jours : une publication compromise est en général détectée et retirée en quelques jours. L'action Sonar 8.4.0, publiée quelques jours avant la mise en place du pipeline, a ainsi été adoptée une semaine après sa publication, par une pull request de Dependabot.

2.2 Scripts d'automatisation

Commande	Objectif	Où elle est définie	Quand elle s'exécute
`./gradlew build`	Compiler, lancer les tests back, produire le JAR et le rapport JaCoCo	`back/build.gradle`	CI (job back) et en local
`./gradlew sonar`	Analyser le back dans SonarQube Cloud	`back/build.gradle` (bloc `sonar`)	CI, si le secret est disponible
`npm ci`	Installer exactement les versions du `package-lock.json`	`front/package-lock.json`	CI (job front)
`npm run test:ci`	Lancer tous les tests front sous Chrome headless, avec couverture lcov	`front/package.json`, `front/karma.conf.js`	CI et en local
`npm run build`	Construire le front en mode production	`front/package.json`, `front/angular.json`	CI et en local
Analyse Sonar du front	Analyser le front dans SonarQube Cloud	`front/sonar-project.properties`	CI, si le secret est disponible
`npm audit signatures`	Vérifier que les paquets installés sont signés par le registre npm	`.github/workflows/ci.yml`	CI (job front)
`docker build --target back` (ou `front`)	Construire l'image d'exécution	`Dockerfile`	CI (job images) et en local

Commande	Objectif	Où elle est définie	Quand elle s'exécute
Trivy `image`	Rapport des vulnérabilités, avec une synthèse dans le récapitulatif du run (§ 5.2)	`.github/workflows/ci.yml`, `.trivyignore.yaml`	CI (job images) et en local
`docker push`	Publier l'image sur GHCR, taguée par le SHA du commit	`.github/workflows/ci.yml`	CI, push sur `main`
`docker buildx imagetools create`	Faire pointer `latest` sur les images testées, sans les reconstruire	`.github/workflows/ci.yml`	CI, après le test de fumée
`docker compose pull`, puis `up --no-build --wait`	Démarrer les images publiées et attendre qu'elles soient saines	`docker-compose.yml`	CI (test de fumée) et sur un serveur

Tableau 4 : Commandes du pipeline

2.3 Reproductibilité

- **Relancer** : onglet *Actions* du dépôt, workflow *CI*, bouton *Run workflow* (`workflow_dispatch`), sur la branche de son choix : c'est ainsi qu'on lance volontairement la CI sur une branche sans pull request ; un job en échec se relance seul avec *Re-run failed jobs*.
- **Reproduire en local** : les mêmes commandes, `./gradlew build` dans `back/` (JDK 21) et `npm ci && npm run test:ci && npm run build` dans `front/` (Node.js 22).
- **Versions figées** : wrapper Gradle, versions de JDK et de Node.js, actions fixées par SHA.
- **Dépendances npm** : `package.json` n'indique que des plages de versions (`^17.3.0` accepte toute version 17.x) ; c'est `package-lock.json` qui fixe les versions exactes, et la CI l'installe tel quel (`npm ci` échoue s'il ne correspond pas à `package.json`). Mais un `npm install` sur un poste peut réécrire ce fichier et y faire entrer une version compromise : toute modification de `package-lock.json` passe donc par une pull request relue, comme le code (risque recensé au § 5.2).
- **Secrets** : deux jetons de projet SonarQube, `SONAR_TOKEN_BACK` et `SONAR_TOKEN_FRONT`, stockés dans les secrets du dépôt GitHub et jamais affichés dans les logs. Chacun ne permet d'analyser que son projet, et n'est transmis qu'à l'étape d'analyse de son job : les commandes de build et de test, qui exécutent aussi le code des dépendances, ne peuvent pas le lire. Sans lui (pull request ouverte depuis un fork ou par Dependabot), l'analyse est sautée et le reste du pipeline s'exécute.

3. Plan de conteneurisation et de déploiement

Ce plan décrit comment l'application est conteneurisée (§ 3.1 et 3.2) et comment ses images sont publiées pour être déployées (§ 3.3).

3.1 Dockerfiles

Le dépôt fournit un seul Dockerfile **multi-stage** : les étapes de build (Node pour le front, Gradle pour le back) produisent les artefacts, et seules des images d'exécution légères sont livrées. Il propose trois cibles.

Cible	Contenu	Usage retenu
front	Caddy sert l'application Angular compilée (ports 80 et 443, HTTPS automatique)	Image publiée et service docker-compose
back	JRE 21 et JAR Spring Boot (port 8080)	Image publiée et service docker-compose

Cible	Contenu	Usage retenu
standalone	Front et back dans un même conteneur, pilotés par supervisor	Non publiée : elle ne reflète pas l'exploitation, qui sépare les deux services

Tableau 5 : Rôle des cibles du Dockerfile existant

Principes que les images doivent respecter :

- **images officielles et minimales**, à version explicite (série fixée : Node 22, Alpine 3.24, Temurin 21 ; pas de tag flottant comme `node` ou `latest`) et encore maintenues ;
- **même image en local et en exploitation** : docker-compose construit les cibles que la CI publie ;
- **utilisateur non-root** dans les images d'exécution ;
- **ports déclarés exacts** (`EXPOSE 8080` pour le back) et **healthcheck** ;
- **aucun secret ni donnée sensible** dans les images ;
- **pas de tests dans le build de l'image** : la CI les a déjà exécutés, l'image ne fait que packager.

Étape	Avant	Après	Taille livrée
Build du front	`node` (sans version)	`node:22-alpine3.24`	non livrée
Build du back	`gradle:jdk17`	`eclipse-temurin:21-jdk-alpine-3.24` et wrapper Gradle du dépôt	non livrée
back	`alpine:3.19` et OpenJDK 21 installé par `apk`	`eclipse-temurin:21-jre-alpine-3.24`	256 Mo (inchangée)
front	`alpine:3.19` et Caddy 2.7.6 installé par `apk`	`alpine:3.24` et Caddy 2.11.7 recompilé	60 Mo (55 Mo avant)

Tableau 6 : Images de base, avant et après

Alpine 3.19 n'est plus maintenue, et le build en JDK 17 ne correspondait pas à l'exécution en Java 21. Corrections apportées au Dockerfile :

- **utilisateurs non-root** (`microcrm`, `caddy`) : Caddy écoute sur les ports 80 et 443 grâce à la seule capacité `cap_net_bind_service` ;
- **correctifs système** : `apk upgrade --no-cache` dans les images d'exécution, car une image de base est figée à sa date de construction ;
- **back** : `EXPOSE 8080` (au lieu de 4200) ; `bootJar` produit le JAR sans relancer les tests ; JAR renommé `microcrm.jar`, sans version en dur ;
- **front** : dépendances npm dans une couche séparée, réutilisée tant que `package-lock.json` ne change pas ; Caddyfile limité à `localhost`.

Caddy recompilé. Au 02/11/2026, la dernière version de Caddy (2.11.7) embarque des bibliothèques Go vulnérables (bibliothèque standard, `x/net`, `go-jose`), corrigées depuis mais pas encore intégrées. Le Dockerfile recompile cette même version avec l'image officielle de build de Caddy (Go 1.27.3) et les plus anciennes versions qui corrigent ces failles. L'image finale part d'Alpine plutôt que de l'image officielle de Caddy, qui contiendrait en plus l'ancien binaire (120 Mo au lieu de 60). Dès qu'une version de Caddy intégrera ces correctifs, on reviendra à l'image officielle.

Partie scannée	Avant	Après
back : système	2 hautes	0
back : dépendances du JAR	8 critiques, 31 hautes	9 critiques, 33 hautes : mêmes dépendances (Spring Boot 3.2.5, § 5), la base de Trivy s'enrichit de nouvelles failles

Partie scannée	Avant	Après
front : système	2 hautes	0
front : Caddy	non analysable (binaire sans informations de version)	1 haute sans correctif, en exception datée

Tableau 7 : Scan Trivy des images (vulnérabilités hautes et critiques)

Le front d'origine paraissait sain parce que Trivy ne pouvait pas lire les dépendances de son binaire Caddy : aucune vulnérabilité détectée ne veut pas dire aucune vulnérabilité.

3.2 docker-compose.yml

docker-compose lance l'application complète avec une seule commande, sur le poste du développeur comme sur un serveur. Il définit deux services :

- `back` : port 8080, healthcheck sur l'API ;
- `front` : ports 80 et 443, démarré seulement quand le back est sain.

Le port 8080 du back reste publié sur la machine : l'adresse de l'API est écrite en dur dans le front (`http://localhost:8080`), c'est donc le navigateur qui appelle le back directement, en HTTP. C'est un risque recensé au § 5.2, avec la solution envisagée.

Le volume `caddy_data` conserve l'autorité de certification locale que Caddy génère pour https://localhost : importée une fois dans le navigateur, elle reste valable quand le conteneur est recréé. Commandes, détaillées dans le README du dépôt :

- `docker compose up --build --wait` : construit les images et rend la main une fois le back sain et le front démarré ;
- `docker compose cp front:/data/caddy/pki/authorities/local/root.crt ~/caddy-root.crt` : extrait l'autorité à importer dans le navigateur ;
- `docker compose down` : arrête les services en conservant les volumes.

3.3 Déploiement

À chaque push validé sur `main`, la CI construit les images, les scanne, les publie sur GitHub Container Registry (GHCR) puis vérifie qu'elles démarrent. Ces images prêtes à l'emploi sont le point de départ de tout déploiement : un serveur les récupère et les lance avec deux commandes :

- `docker compose pull` : télécharge les dernières images validées (`latest`) ;
- `docker compose up -d --no-build --wait` : redémarre les services avec ces images et attend qu'ils soient sains.

Étape	Objectif
Build des images front et back	Produire exactement ce qui sera exécuté
Scan des vulnérabilités (Trivy)	Recenser les vulnérabilités hautes et critiques de chaque image avant de la publier
Publication sur GHCR	Tag : SHA du commit, pour savoir exactement quel code tourne et y revenir
Test de fumée	Démarrer les images publiées et interroger l'API et le front
Promotion de `latest`	Ne désigner comme version courante que des images qui ont passé le test de fumée

Tableau 8 : Chaîne de déploiement

Fonctionnement dans le pipeline :

- **Build et scan à chaque run** (pull request, `main`, nuit) : le rapport et sa synthèse recensent les

vulnérabilités de chaque image, et le run de nuit révèle une vulnérabilité publiée après coup sur une image inchangée, sans bloquer la livraison ;

- **publication sur push vers `main` seulement** : images `ghcr.io/manooweb/microcrm-back` et `microcrm-front` ;
- **aucun secret à créer** : la publication utilise le jeton fourni par GitHub Actions à chaque run (`GITHUB_TOKEN`), avec le droit d'écrire des paquets donné aux seuls jobs qui publient les images et déplacent `latest` ;
- **test de fumée** : démarre avec `docker-compose.yml` les images que le run vient de publier (tag du SHA), vérifie leur healthcheck, puis interroge l'API et le front en HTTPS ;
- **promotion** : si le test de fumée réussit, `latest` est déplacé sur ces images, à l'identique (même digest) ; s'il échoue, `latest` reste sur la dernière version validée, et l'image défaillante n'est récupérée que si on demande explicitement son tag.

Retour à une version précédente : chaque image publiée garde le SHA de son commit comme tag. Pour revenir en arrière, on relance les services sur ce tag : `MICROCRM_TAG=<sha> docker compose up -d --no-build --wait`, après un `docker compose pull` avec la même variable.

Aucune plateforme d'hébergement n'est définie pour MicroCRM : le déploiement automatisé s'arrête donc à des images publiées et vérifiées, que n'importe quel serveur doté de Docker peut lancer avec ces commandes.

Prérequis du serveur

Prérequis	Détail
Docker Engine et le plugin Compose	`docker compose version` doit répondre ; aucun JDK ni Node.js : tout est dans les images
Accès sortant à `ghcr.io`	Images publiques : aucun identifiant à configurer pour les télécharger
Ports libres	80 et 443 (front), 8080 (API, appelée directement par le navigateur)
Mémoire	Environ 400 Mo pour les deux conteneurs (back 350 Mo, front 55 Mo, mesurés au repos avec `docker stats`)
Fichier `docker-compose.yml`	Celui du dépôt, à la même version que les images : il porte les ports, les volumes, les healthchecks et l'ordre de démarrage

Tableau 9 : Prérequis techniques d'un déploiement

Ordre de déploiement

Étape	Commande ou action	Ce qui se passe
1. Choisir la version	`latest` par défaut, ou `MICROCRM_TAG=<sha>`	`latest` désigne les dernières images qui ont passé le test de fumée
2. Télécharger les images	`docker compose pull`	Le serveur ne compile rien : il récupère les images construites et testées par la CI
3. Démarrer	`docker compose up -d --no-build --wait`	Le back démarre d'abord ; le front attend que le healthcheck du back soit au vert, puis démarre à son tour
4. Vérifier	`docker compose ps`, puis l'API (`/persons`) et https://localhost	Les deux services doivent être `healthy` ; sinon, retour à la version précédente (§ 7.3)

Tableau 10 : Déploiement pas à pas

Risques à la mise en production

Les incidents les plus probables d'un déploiement, et ce que la chaîne prévoit pour chacun :

Scénario	Prévention ou détection	Reprise
Image défaillante publiée, ou promotion oubliée	Promotion automatique de `latest`, seulement après le test de fumée	Rien à faire : `latest` reste sur la dernière version validée
Dépendance externe indisponible (Maven Central, npm, Docker Hub)	Le build échoue en CI et rien n'est publié	Relancer le run ; le serveur n'en dépend pas, il ne télécharge que des images déjà construites
Registre GHCR inaccessible au déploiement	`docker compose pull` échoue ; les conteneurs en place continuent de tourner	Relancer plus tard ; `up` sans `pull` redémarre la version déjà présente sur le serveur
Configuration minimale non respectée (port occupé, mémoire)	`up --wait` échoue et indique le service en cause	Libérer la ressource (prérequis ci-dessus), puis relancer
Nouvelle variable d'environnement non reportée	Les valeurs par défaut sont dans l'image (`SPRING_PROFILES_ACTIVE`) ou dans le `docker-compose.yml` versionné (`MICROCRM_TAG`)	Déployer le `docker-compose.yml` de la même version que les images
Migration du schéma de la base oubliée	Sans objet aujourd'hui : la base est en mémoire et son schéma est recréé à chaque démarrage	Avec une base persistante : migrations versionnées (Flyway) appliquées au démarrage, après une sauvegarde (§ 7)

Tableau 11 : Scénarios d'incident au déploiement

4. Plan de testing périodique

Ce plan définit quand, comment et dans quel but les tests s'exécutent automatiquement. Il s'applique à l'application existante.

4.1 Types de tests automatisés

Test	Outil	Ce qu'il vérifie
Tests back	JUnit 5 via Gradle, couverture JaCoCo	Démarrage du contexte Spring et requête du dépôt JPA sur HSQLDB
Tests front	Karma et Jasmine sous Chrome headless, couverture lcov	Instanciation des composants et des services, titre de l'application
Build	`./gradlew build`, `ng build`	Le code compile et se package
Analyse statique	SonarQube Cloud, quality gate	Qualité, sécurité et couverture du code
Test de fumée des images	`docker compose up --wait` puis `curl`	Les images publiées démarrent et répondent

Tableau 12 : Tests exécutés par le pipeline

> Les tests fournis sont des **tests de fumée** : 2 tests back et 8 tests front, dont 6 vérifient seulement qu'un composant s'instancie. Le pipeline les exécute, mais ils ne prouvent pas le comportement métier. La couverture affichée par SonarQube sera donc faible ; c'est une dette identifiée, traitée dans le plan d'action.

4.2 Fréquence d'exécution

Déclencheur	Ce qui s'exécute	Pourquoi
Pull request vers `main` (ouverte, puis à chaque push sur sa branche)	Build, tests, couverture, analyse SonarQube, build et scan des images ; la fusion est bloquée si le build ou les tests échouent	Retour rapide au développeur ; rien n'arrive sur `main` sans être validé
Push sur `main` (après fusion)	Tout, plus publication, test de fumée et promotion des images	Ne livrer que du code validé
Chaque nuit (`schedule`)	Sur `main` : build, tests, analyse SonarQube, build et scan des images, sans publication	Détecter ce qui casse sans nouveau commit : nouvelle vulnérabilité, image de base ou dépendance modifiée, nouvelle règle Sonar
À la demande (`workflow_dispatch`)	Comme la nuit, sur la branche choisie	Relancer après un incident ou une correction externe

Tableau 13 : Déclencheurs du pipeline

4.3 Objectifs des tests

- **Non-régression** : chaque modification est testée avant d'atteindre `main`.
- **Validation fonctionnelle** : l'application démarre et répond (contexte Spring, composants, test de fumée des images).
- **Qualité** : la quality gate SonarQube signale, sur chaque pull request, le nouveau code qui dégrade la fiabilité, la sécurité ou la maintenabilité.
- **Vérification avant déploiement** : seules les images issues d'un pipeline vert sont publiées, et `latest` ne désigne que des images qui ont passé le test de fumée.

5. Plan de sécurité

5.1 Résultats SonarQube

Résultats de l'analyse SonarQube Cloud du 05/11/2026 sur `main` (commit `5179904`), avec le profil de règles par défaut de l'organisation. Notes de A (meilleure) à E.

Indicateur	Back (Java)	Front (Angular)
Lignes de code	226	733
Vulnérabilités (note de sécurité)	3 (C)	1 (B)
Bugs (note de fiabilité)	0 (A)	25 (C)
Code smells (note de maintenabilité)	8 (A)	37 (A)
Security hotspots à relire	0	0
Duplication	9,2 % (2 blocs identiques)	0 %
Complexité cyclomatique / cognitive	33 / 3	58 / 15
Couverture des tests	56,4 % (2 tests)	27,0 %
Quality gate	Réussie	Réussie

Tableau 14 : Synthèse de l'analyse SonarQube

La quality gate réussit malgré une note de sécurité C : elle ne juge que le **nouveau code**, c'est-à-dire ce qui change depuis l'analyse de référence. La dette existante ne la fait donc pas échouer ; elle est suivie par ce plan et non par la gate. Sur les 74 alertes, 52 sont prioritaires et présentées par règle ci-dessous ;

les 22 autres sont mineures. Toutes sont reprises dans le plan d'action (§ 5.3).

Le tableau compte les alertes **par type** (vulnérabilité, bug, code smell). La vue d'ensemble de SonarQube Cloud (annexes, § 10.3) les compte **par qualité touchée**, et une même alerte peut en toucher plusieurs : les 2 alertes de fiabilité qu'elle affiche pour le back sont des code smells qui touchent aussi la fiabilité, d'où 0 bug dans le tableau.

Vulnérabilités

Une vulnérabilité est une faille exploitable ; elle se traite avant les bugs et les code smells, qui touchent la fiabilité et la lisibilité du code sans ouvrir de porte à un attaquant.

Règle	Où	Constat	OWASP Top 10 2025
`java:S5122` (×3)	`SpringDataRestCustomization`, `PersonRepository`, `OrganizationRepository`	CORS ouvert : n'importe quel site peut appeler l'API depuis le navigateur de l'utilisateur	A02 : mauvaise configuration
`Web:S5725` (×1)	`front/src/index.html`	Feuille de style Bulma chargée depuis un CDN sans contrôle d'intégrité : un CDN compromis pourrait modifier la page	A08 : intégrité des logiciels

Tableau 15 : Vulnérabilités relevées

Bugs et code smells critiques

Règle	Où	Constat
`typescript:S9383` (×16)	Composants du front	Promesses sans gestion d'erreur : si l'API échoue, l'utilisateur n'est pas prévenu et l'erreur se perd
`typescript:S4123` (×14)	`person.service.ts`, `organization.service.ts`	`await` appliqué à un Observable, sans effet : le vrai point d'attente est `firstValueFrom`, le code induit en erreur
`typescript:S7059` (×1)	`person-details.component.ts`	Chargement asynchrone dans le constructeur, au lieu de `ngOnInit`
Accessibilité (×15) : `Web:InputWithoutLabelCheck`, `Web:S6853`, `Web:S5256`, clic sans clavier	Formulaires et tableau des fiches	Champs sans libellé associé, tableau sans en-têtes : formulaires inutilisables avec un lecteur d'écran
`java:S6813` (×2)	`InitialDataFixture`	Injection par champ au lieu du constructeur : classe plus difficile à tester

Tableau 16 : Bugs et code smells critiques

Le reste (22 alertes) est mineur : membres à marquer `readonly`, imports inutilisés, fichiers CSS vides, `return` superflus, `Number.parseInt`, API `java.time`. Ces corrections sont mécaniques et regroupées dans un lot de nettoyage.

Zones de complexité

La complexité cyclomatique compte les chemins possibles dans le code ; la complexité cognitive mesure l'effort pour le lire. Ainsi `Person.java` a une complexité cyclomatique de 17 mais cognitive de 1 : ce sont des accesseurs, simples à lire. Aucune fonction ne dépasse le seuil d'alerte de SonarQube (15 en complexité cognitive). Les zones à surveiller sont celles qui cumulent logique et faible couverture :

Fichier	Cyclomatique	Cognitive	Couverture
`person-details.component.ts`	21	7	26,7 %
`organization-details.component.ts`	11	4	36,0 %
`organization.service.ts`	9	2	12,5 %
`person.service.ts`	7	2	14,8 %

Tableau 17 : Fichiers les plus complexes

Duplications

Côté back, les 9,2 % de duplication sont un même bloc de 14 lignes, répété dans `Person` et `Organization` (Sonar compte 2 blocs) : les dates de création et de modification et leurs accesseurs. Une classe parente commune (`@MappedSuperclass`) le supprimerait. Le front n'a aucune duplication.

Couverture des tests

La couverture du back (56,4 %) est trompeuse : un seul test démarre toute l'application, données de démonstration comprises, et exécute le code sans vérifier son comportement. Celle du front (27 %) est faible, surtout dans les services qui appellent l'API (12 à 15 %). La quality gate exige 80 % de couverture sur le nouveau code : toute évolution devra donc être livrée avec ses tests (§ 4).

Croisement avec les logs

Les logs collectés par ELK (§ 6.1) révèlent un défaut que l'analyse statique ne voit pas : une requête sur `/persons/abc` ou `/persons/NaN` renvoie une **erreur 500** au lieu d'un 400. Le back ne valide pas le format de l'identifiant. Côté front, `parseInt` sans contrôle (`typescript:S7773`, signalé comme mineur) peut produire `NaN` et déclencher cette erreur. Et comme les promesses ne gèrent pas les erreurs (`S9383`), l'utilisateur ne voit rien. Une alerte mineure pour SonarQube devient ainsi prioritaire une fois rapprochée des logs.

5.2 Analyse des risques

Rôle de SonarQube Cloud

SonarQube Cloud analyse le back et le front, chacun dans son projet, à chaque push sur `main` et à chaque pull request. La quality gate « Sonar way » s'applique au nouveau code. Son verdict apparaît sur chaque run et chaque pull request, sans bloquer la fusion : le projet existant porte une dette (tests limités, composants sans support) qui n'est pas corrigée dans le cadre de cette industrialisation. Elle est recensée ci-dessous et planifiée dans le plan d'action (§ 5.3).

Catégorie	Exemple
Vulnérabilités	Injection, configuration de sécurité permissive
Security hotspots	Code sensible à relire par un humain (CORS, cryptographie, entrées utilisateur)
Bugs (fiabilité)	Valeur nulle non gérée, ressource non fermée
Code smells (maintenabilité)	Duplication, complexité excessive, code mort
Couverture	Part du code exécutée par les tests (JaCoCo et lcov)

Tableau 18 : Problèmes surveillés par SonarQube

Risques identifiés

Les risques suivants sont recensés avec la solution envisagée ; leur priorité est fixée dans le plan d'action (§ 5.3). Le code de l'application n'est pas modifié pour les corriger : seules les images Docker

ont été corrigées (§ 3.1). La chaîne CI/CD les rend visibles à chaque run, par SonarQube, Trivy et Dependabot. Les références renvoient à l'OWASP Top 10 2025.

Risque	Où	Référence	Solution envisagée
CORS ouvert à toutes les origines : `allowedOrigins("*")`	`SpringDataRestCustomization`	OWASP A02 : mauvaise configuration	Limiter aux origines du front ; inutile si l'API passe par Caddy, sur la même origine que le front (voir « API appelée en HTTP »)
Aucune authentification : lecture, création et suppression libres	API Spring Data REST	OWASP A01 : contrôle d'accès	Authentification (Spring Security) avant toute exposition hors du poste local
API appelée en HTTP, port 8080 publié sur la machine	`front/src/app/config.ts`, `docker-compose.yml`	OWASP A04 : défaillances cryptographiques	Caddy seul point d'entrée en HTTPS : il transmet les appels d'API au back, le front utilise une adresse relative, le port 8080 n'est plus publié
Composants sans support : Spring Boot 3.2.5, Angular 17 (support terminé le 15/05/2025)	`back/build.gradle`, `front/package.json`	OWASP A03 : chaîne d'approvisionnement logicielle	Montée de version (§ 8). Trivy relève 9 critiques et 33 hautes dans le JAR ; les mises à jour du front proposées par Dependabot échouent, Angular 17 imposant TypeScript 5.4 au plus
Plages de versions npm (^, ~) : un `npm install` local peut faire entrer une version compromise	`front/package.json`	OWASP A03 : chaîne d'approvisionnement logicielle	Versions exactes dans `package.json` ; en attendant, `npm ci`, pull requests relues, `npm audit signatures`, délai de Dependabot
Tests limités à des tests de fumée (§ 4.1)	`back/src/test`, `front/src/app`	Qualité	Tests du comportement de l'API et des services front
Images : Alpine 3.19 sans support, exécution en root, tag flottant	Dockerfile	Bonnes pratiques Docker	Corrigé (§ 3.1)

Tableau 19 : Risques identifiés dans l'application existante

Risques liés au pipeline

Chaque risque de la chaîne CI/CD est traité par une mesure ; le risque qui subsiste est repris dans le plan d'action (§ 5.3).

- **Secrets** : uniquement dans GitHub Secrets (un jeton SonarQube par projet) ; GHCR utilise le `GITHUB_TOKEN` fourni par GitHub. Aucun secret dans le code, les images ou les logs.
- **Permissions minimales** : lecture seule par défaut ; écriture sur les packages seulement pour les jobs qui publient les images et déplacent `latest`, écriture dans le dépôt seulement pour le workflow Métriques, sur sa branche dédiée (§ 6.2).
- **Actions** : actions officielles uniquement, **fixées par SHA de commit** (version en commentaire) plutôt que par tag. Un tag peut être déplacé : le 19 mars 2026, des attaquants ont réécrit 76 des 77 tags de l'action Trivy pour voler les secrets des pipelines qui l'utilisaient (CVE-2026-33634, avis GHSA-69fq-xp46-6x23 d'Aqua Security). Un SHA, lui, désigne un code qui ne peut plus changer.
- **Dépendances** : Dependabot surveille chaque semaine Gradle, npm, les images Docker et les actions GitHub, dont il met à jour les SHA et leur commentaire de version. Il attend 7 jours avant de proposer une version tout juste publiée, et écarte les montées de version majeure, qui relèvent du plan de mise à jour ; ses mises à jour de sécurité, elles, n'attendent pas ce délai. Ses pull requests ne sont pas fusionnées d'office : elles sont examinées dans le plan de mise à jour (§ 8).
- **Images** : scan des images avec Trivy, scanner open source et gratuit, avant publication. Son rapport recense les vulnérabilités hautes et critiques de chaque image (système et dépendances applicatives), avec une synthèse dans le récapitulatif de chaque run, sans bloquer la publication. Une

faille acceptée faute de correctif est inscrite dans `.trivyignore.yaml`, justifiée et datée. Le système des images est mis à jour à chaque build (`apk upgrade`) ; les dépendances de Spring Boot 3.2.5 relèvent du plan de mise à jour.

Risque	Ce qui subsiste
Fuite d'un jeton SonarQube	Les jetons n'ont pas de date d'expiration ni de renouvellement planifié
Image vulnérable publiée	Trivy rend compte sans bloquer : l'image du back est publiée avec 9 failles critiques connues, faute de montée de version
Contrôles non bloquants ignorés	Un verdict rouge de SonarQube ou de Trivy n'arrête rien : il doit être lu
Mise à jour d'action non relue	Une pull request de Dependabot fusionnée sans relecture ferait entrer le code d'un tiers dans la CI
Runs planifiés non garantis	GitHub peut retarder ou sauter un run planifié en période de forte charge : le scan nocturne n'est pas certain

Tableau 20 : Risques résiduels du pipeline

5.3 Plan d'action et remédiation

Les actions suivent la gravité : la sécurité d'abord, puis la fiabilité, puis la maintenabilité. Elles ne sont pas réalisées dans le cadre de l'industrialisation, qui ne modifie pas le code de l'application ; elles forment le plan proposé pour les prochaines itérations.

Échéance	Action	Alertes ou risques traités
Immédiat	Restreindre le CORS aux origines du front	`java:S5122` (×3)
Immédiat	Installer Bulma par npm (ou ajouter l'attribut `integrity`)	`Web:S5725`
Immédiat	Valider les identifiants : 400 côté back, contrôle de `parseInt` côté front	Erreurs 500 vues dans ELK, `typescript:S7773` (×2)
Immédiat	Donner une date d'expiration aux jetons SonarQube et planifier leur renouvellement	Fuite de jeton
Court terme	Gérer les erreurs des promesses et prévenir l'utilisateur ; charger les données dans `ngOnInit`	`typescript:S9383` (×16), `S7059`
Court terme	Retirer les `await` sans effet dans les services	`typescript:S4123` (×14)
Court terme	Écrire des tests de comportement : API (MockMvc), services front (`HttpTestingController`)	Couverture, risque « tests limités » (§ 5.2)
Court terme	Associer libellés et champs, ajouter les en-têtes du tableau	Accessibilité (×15)
Court terme	Injection par constructeur ; classe parente pour les dates	`java:S6813` (×2), duplication
Court terme	Lot de nettoyage mécanique	22 alertes mineures
Court terme	Revue hebdomadaire des verdicts SonarQube et Trivy et des pull requests de Dependabot	Contrôles ignorés, mises à jour non relues
Long terme	Montée de version de Spring Boot et d'Angular (§ 8), passage à `java.time`	9 failles critiques et 33 hautes (Trivy), `java:S2143`
Long terme	Authentification (Spring Security) ; Caddy seul point d'entrée en HTTPS	Risques « authentification » et « API en HTTP » (§ 5.2)
Long terme	Durcir les contrôles une fois la dette résorbée : quality gate et Trivy bloquants, condition sur la note de sécurité globale	Contrôles non bloquants, image vulnérable publiée

Tableau 21 : Plan d'action

6. Monitoring, métriques et KPI

6.1 Mise en place du monitoring

Le monitoring centralise les logs du back et du front pour suivre les erreurs, les tendances et les performances de l'application. Il repose sur la stack ELK (Elasticsearch, Logstash, Kibana) et tourne sur le poste de développement, à côté de l'application lancée avec Docker Compose (§ 3.2). Il reste hors de la chaîne CI/CD : la stack demande environ 4 Go de mémoire et n'a rien à surveiller dans un run de quelques minutes.

Architecture

La stack est décrite dans un fichier Compose distinct, `docker-compose-elk.yml`, qui forme son propre projet (`microcrm-elk`) : démarrer ou arrêter l'un ne touche pas l'autre. Les quatre composants utilisent la version 8.19.24, dernière de la série 8 ; le passage à la série 9 relève du plan de mise à jour (§ 8).

Service	Rôle	Mémoire maximale
Filebeat	Lit les fichiers de logs que Docker écrit pour chaque conteneur et ne garde que ceux des images MicroCRM	256 Mo
Logstash	Décode les lignes JSON, nomme les champs, écarte les requêtes des healthchecks (`elk/logstash/logstash.conf`)	1 Go
Elasticsearch	Stocke les événements, un index par jour (`microcrm-logs-AAAA.MM.JJ`)	2 Go
Kibana	Affiche le tableau de bord, importé automatiquement au démarrage (`elk/kibana/tableau-de-bord.ndjson`)	1 Go

Tableau 22 : Services de la stack de monitoring

Filebeat lit les logs déjà écrits par Docker : l'application n'a pas besoin de connaître ELK et continue de fonctionner si la stack est arrêtée. L'autre solution, un envoi direct du back vers Logstash, aurait lié l'application à son monitoring.

Sources de logs

Chaque source écrit une ligne JSON par événement, que Logstash décode sans expression régulière. Les formats sont activés dans les images : le back passe en JSON grâce au profil Spring `docker`, défini dans `application-docker.properties` ; en développement (`./gradlew bootRun`), il garde son format texte, plus lisible.

Source	Production du JSON	Contenu utile
Logs applicatifs du back	Encodeur JSON intégré à Logback (`logback-json.xml`), sans dépendance ajoutée	Niveau (INFO, WARN, ERROR), classe émettrice, message, exception
Requêtes de l'API	Journal des requêtes de Tomcat, écrit sur la sortie standard	Méthode, chemin, code HTTP, durée en millisecondes
Requêtes du front	Journal des requêtes de Caddy (directive `log` du Caddyfile)	Méthode, chemin, code HTTP, durée

Tableau 23 : Sources de logs collectées

Logstash range ces informations dans des champs communs aux deux services, nommés selon la convention ECS (Elastic Common Schema) : `service.name`, `log.level`, `http.response.status_code`, `url.path`. Un modèle d'index les déclare en valeurs exactes, pour que

Kibana puisse les compter et les regrouper.

Tableau de bord

Panneau	Question à laquelle il répond
Volume de logs par service	L'activité est-elle normale ? Un pic ou un silence signale un problème.
Erreurs et avertissements	Combien de réponses 4xx et 5xx et de logs WARN ou ERROR, et quand ?
Durée des requêtes de l'API	L'API ralentit-elle ? Moyenne et 95e centile, en millisecondes.
Codes de réponse HTTP	Quelle part des requêtes échoue, et de quelle façon (404, 500) ?
Requêtes par famille de codes HTTP	Quelle part des requêtes réussit, échoue côté client (4xx, erreur fonctionnelle) ou côté serveur (5xx, erreur technique) ?
Dernières erreurs	Le détail des derniers événements en erreur : service, code, chemin, message.

Tableau 24 : Panneaux du tableau de bord Kibana



Figure 2 : Tableau de bord Kibana après une séance de trafic de démonstration

Le script `elk/generer-trafic.sh` produit ce trafic : des requêtes normales, des 404 (ressource absente) et des 500 (identifiant invalide), pour vérifier que chaque panneau réagit.

Mise en œuvre

Commande	Effet
`docker compose -f docker-compose-elk.yml up -d`	Démarre la stack ; le tableau de bord est importé dès que Kibana est prêt (environ une minute)
`elk/generer-trafic.sh 60`	Envoie 60 séries de requêtes à l'application
`docker compose -f docker-compose-elk.yml down`	Arrête la stack en conservant les logs indexés (`down -v` les supprime)

Tableau 25 : Commandes du monitoring

Kibana est ensuite accessible sur `http://localhost:5601`, tableau de bord « MicroCRM – logs de l'application ».

Choix de sécurité et limites

- **Sécurité d'Elasticsearch désactivée** (ni mot de passe ni TLS) pour garder une configuration minimale : les ports 9200 et 5601 ne sont ouverts que sur `127.0.0.1`, donc inaccessibles depuis le réseau. Une stack partagée ou exposée devrait activer l'authentification et le chiffrement.
- **Erreurs du navigateur non collectées** : une erreur JavaScript du front Angular reste dans la console

du navigateur. Les collecter demanderait d'envoyer ces erreurs à un service dédié.

- **Appels de l'API hors de Caddy** : le navigateur appelle directement le back (§ 5.2, « API appelée en HTTP ») ; ces requêtes n'apparaissent que dans le journal du back.
- **Pas de durée de conservation** : un index est créé chaque jour et rien ne le supprime. Sur un poste de développement, `down -v` suffit ; une stack durable demanderait une politique de cycle de vie des index.
- **Pas d'alerte** : le tableau de bord se consulte, il ne prévient pas. Des alertes Kibana sur le taux de réponses 5xx seraient l'étape suivante ; elles demandent une clé de chiffrement et la sécurité d'Elasticsearch. Avec la configuration minimale, les menus qui en dépendent (alertes, Security, Cases) affichent des erreurs ; le tableau de bord et l'exploration des logs n'en dépendent pas.

6.2 Métriques DORA

Les métriques DORA mesurent la performance de livraison : la vitesse (fréquence, délai) et la stabilité (échecs, rétablissement). Elles sont calculées à partir de l'historique GitHub Actions du dépôt par le script `scripts/metriques-ci.mjs` (JavaScript, sans dépendance), qui lit l'API de GitHub et produit un rapport en Markdown. Le workflow `Métriques` le lance après chaque run CI sur `main`, chaque lundi et à la demande : il affiche le rapport dans le récapitulatif du run et publie les quatre valeurs DORA en badges, en tête du README du dépôt. Ce workflow est le seul autorisé à écrire dans le dépôt, et seulement sur une branche dédiée (`metriques`) : ses publications ne relancent pas la CI et ne faussent donc pas le calcul. Les métriques décrivent la chaîne et l'équipe, pas les personnes : elles servent à repérer où la livraison ralentit, pas à évaluer quelqu'un.

MicroCRM n'a pas de plateforme d'exploitation (§ 3.3) : sa « production » est le registre GHCR, où le pipeline publie les images validées. Les définitions sont adaptées en conséquence :

Métrique	Définition retenue	Calcul
Fréquence de déploiement	Un déploiement est un run sur `main` qui publie les images, les valide par le test de fumée et déplace `latest`	Nombre de déploiements divisé par la durée de la période, depuis la mise en place du déploiement continu
Délai de livraison	Du premier commit d'une pull request à la fin du déploiement de sa fusion	Médiane sur les pull requests fusionnées et déployées ; décomposée en attente avant la PR, revue, puis pipeline
Taux d'échec des changements	Un changement est un push sur `main` ; il échoue si son run échoue, relances comprises	Changements en échec divisés par le nombre de changements
Temps de rétablissement (MTTR)	Un incident est un run rouge déclenché par un push sur `main` ; il prend fin au run vert suivant. Un échec de nuit n'est pas un incident de livraison	Moyenne des durées entre la fin du run en échec et la fin du run réussi suivant

Tableau 26 : Définition et calcul des métriques DORA

Métrique	Valeur	Base de calcul
Fréquence de déploiement	1,5 par jour	11 déploiements en 7,6 jours (03/11 au 10/11)
Délai de livraison (médiane)	1,1 h	11 pull requests, de 10,9 min à 17,4 h
Taux d'échec des changements	0 % depuis le déploiement continu	0 sur 11 ; 14 % sur tout l'historique (2 sur 14), pendant la mise en place de la CI
Temps de rétablissement	1,6 h	1 incident : clés de projet SonarQube erronées, corrigées par une pull request

Tableau 27 : Valeurs relevées le 10/11/2026

Le délai de livraison se décompose ainsi : le pipeline (de la fusion au déploiement) prend environ 4 minutes et la revue de 4 minutes à 1,2 heure ; l'essentiel du délai, jusqu'à 17 heures, s'écoule **avant l'ouverture de la pull request**. Le délai est compté en heures calendaires : il inclut les nuits et le temps consacré à d'autres sujets, l'équipe ne travaillant pas à plein temps sur MicroCRM.

6.3 KPI personnalisés

Les KPI complètent DORA par des mesures opérationnelles. Ils viennent de deux sources à ne pas confondre : le pipeline (GitHub Actions, même script) et l'application (logs collectés par ELK, § 6.1). Les cibles sont proposées pour suivre la tendance ; elles ne bloquent rien.

KPI	Source et calcul	Valeur	Cible proposée
Durée du pipeline	GitHub : médiane de la durée des runs de déploiement (11) et des runs de pull request (13)	4,3 min (déploiement), 3,7 min (PR)	Moins de 10 min
Temps de build et de tests	GitHub : médiane des jobs back (42) et front (36), et de leur étape de tests	Back 1,0 min dont 0,4 min de build et tests ; front 1,4 min dont 0,3 min de tests	Suivre la tendance : une hausse signale des tests ou dépendances qui alourdissent le build
Taux de réussite des runs de pull request	GitHub : runs de PR réussis, séparés entre l'équipe et Dependabot	Équipe 100 % (13) ; Dependabot 63 % (16)	90 % pour Dependabot
Qualité SonarQube	SonarQube : vulnérabilités et bugs de la dernière analyse (§ 5.1)	Back : 3 vulnérabilités, 0 bug ; front : 1 vulnérabilité, 25 bugs	0 vulnérabilité ; note A partout
Taux d'erreurs de l'application	ELK : réponses 5xx divisées par les requêtes HTTP	3,1 % (99 sur 3 191) ; 4xx : 6,7 %	Moins de 1 % de 5xx

Tableau 28 : KPI du pipeline et de l'application

Les valeurs d'ELK portent sur du trafic de démonstration (§ 6.1), qui provoque volontairement des 404 et des 500 : elles valident la méthode de calcul, pas le comportement d'utilisateurs réels. La durée des requêtes de l'API, suivie dans le tableau de bord, reste faible : 1 ms en médiane et 3,5 ms au 95e centile, calculés dans Elasticsearch sur la même période.

6.4 Analyse synthétique

Tendances et points forts

- **Le pipeline n'est pas un frein** : 4 minutes pour livrer, et le test de fumée garantit que `latest` ne désigne que des images qui démarrent.
- **Livraison fréquente et stable depuis le déploiement continu** : 11 déploiements, aucun échec. Les deux échecs de l'historique datent de la mise en place de la CI et ont été corrigés en 1,6 heure.
- **Exécutions planifiées fiables** : 9 exécutions nocturnes sur 10 ont réussi (rapport du workflow Métriques). L'échec, sans modification du code, venait du build de l'image back et a réussi à la relance : c'est le rôle du run de nuit de révéler ces changements extérieurs au code.
- **Le délai de livraison dépend de la disponibilité de l'équipe, pas de la chaîne** : le pipeline n'en représente que 4 minutes, le reste suit le rythme de travail sur le projet.

Points à améliorer

- **37 % des runs de CI des pull requests de Dependabot échouent** : les mises à jour du front butent sur Angular 17 (TypeScript 5.4 au plus). Ce KPI mesure la dette de versions ; il remontera avec la montée de version (§ 8).

- **Les erreurs 500 viennent d'un défaut de validation** (§ 5.1) : leur correction fera baisser le taux d'erreurs, que le tableau de bord permettra de vérifier.
- **Base de mesure courte** : 7,6 jours de déploiement continu et une équipe d'une personne. Les valeurs donnent un point de départ ; la tendance n'aura de sens qu'après plusieurs semaines.

Tableaux de bord et alertes

Deux tableaux de bord couvrent les deux sources : le tableau de bord Kibana pour l'application (§ 6.1) et le workflow `Métriques` pour le pipeline, dont les badges du README donnent l'état à jour. Aucune alerte automatique n'est en place. Les premières à créer : un run rouge sur `main` (notification GitHub, déjà active pour l'auteur du commit) et un taux de 5xx supérieur à 1 % (alerte Kibana, qui demande d'activer la sécurité de la stack).

7. Plan de sauvegarde des données

MicroCRM n'a aujourd'hui aucune donnée métier durable : sa base HSQLDB est en mémoire et un jeu de démonstration est recréé à chaque démarrage. Ce qui doit pouvoir être restauré, c'est donc avant tout une **version qui fonctionne** : son code, sa configuration et ses images. La plupart de ces sauvegardes sont déjà assurées par la chaîne CI/CD, sans action manuelle.

7.1 Ce qui doit être sauvegardé

Élément	Où il se trouve	Sauvegarde	Fréquence
Code, Dockerfile, `docker-compose.yml`, Caddyfile, configuration Spring, workflows	Dépôt Git sur GitHub	Historique Git, plus chaque clone de développeur	À chaque push
Images du back et du front	GHCR, une image par commit de `main` (tag du SHA)	Conservées dans le registre, sans suppression automatique	À chaque déploiement
Autorité de certification de Caddy	Volume `caddy_data` du serveur	Archive du volume (§ 7.2)	À la création, puis après chaque régénération
Données de l'application	Base HSQLDB en mémoire	**Aucune** : perdues à chaque redémarrage, par conception du projet	Sans objet aujourd'hui
Secrets (jetons SonarQube)	GitHub Secrets	Aucune copie, volontairement : un jeton perdu se régénère dans SonarQube Cloud	Sans objet

Tableau 29 : Éléments à sauvegarder

Les logs collectés par ELK et les métriques du pipeline ne sont pas sauvegardés : les premiers sont des données de diagnostic du poste de développement, les secondes se recalculent à partir de l'historique de GitHub Actions.

7.2 Procédure de sauvegarde

Le code et les images sont sauvegardés automatiquement : par Git à chaque push, et par la CI à chaque déploiement, qui publie une image identifiée par son SHA. Seul le volume de Caddy demande une sauvegarde manuelle, à lancer sur le serveur. Elle archive le volume entier dans un dossier de sauvegarde **hors du dépôt** (`~/sauvegardes`) :

Action	Commande
Créer le dossier de sauvegarde (une fois)	`mkdir -p ~/sauvegardes`
Archiver le volume (lecture seule)	`docker run --rm -v projet7_caddy_data:/data:ro -v ~/sauvegardes:/sauvegarde alpine:3.24 tar czf /sauvegarde/caddy_data-$(date +%F).tar.gz -C /data .`

Tableau 30 : Sauvegarde du volume de Caddy

Un conteneur Alpine éphémère lit le volume (monté en `/data`) et écrit l'archive dans `/sauvegarde`, qui est le dossier `~/sauvegardes` du serveur. L'archive part de la racine du volume (`./`) : la restauration redonne le volume à l'identique, droits compris. L'essentiel en est l'autorité de Caddy (`caddy/pki/authorities/local/`, certificat `root.crt` et clé `root.key`) ; le reste, Caddy le régénère à partir d'elle.

L'archive pèse quelques kilo-octets. Elle contient la clé privée de l'autorité de certification : elle se range hors du dépôt, avec les mêmes précautions qu'un secret. Le nom du volume (`projet7_caddy_data`) reprend celui du dossier du projet Compose.

Si la base devient persistante (PostgreSQL par exemple), ce plan s'enrichira d'une sauvegarde quotidienne de la base (`pg_dump`), conservée hors du serveur, et d'une sauvegarde juste avant chaque migration de schéma.

7.3 Procédure de restauration

Chaque incident a sa procédure ; toutes s'appuient sur les sauvegardes ci-dessus.

Incident	Procédure	Commandes
Version défectueuse en service	Revenir à la version précédente : retrouver son SHA (historique de `main` ou page du paquet sur GHCR), puis relancer les services sur ce tag	`MICROCRM_TAG=<sha> docker compose pull` puis `MICROCRM_TAG=<sha> docker compose up -d --no-build --wait`
Correction définitive du défaut	Annuler le commit fautif sur `main` par une pull request ; la CI reconstruit, teste et promeut automatiquement la version corrigée	`git revert <sha>`, puis pull request ; ensuite `docker compose pull` et `up` sur le serveur
Conteneur arrêté ou bloqué	Docker relance seul un conteneur arrêté (`restart: unless-stopped`) ; un conteneur bloqué (`unhealthy`) se relance à la main, après lecture de ses journaux	`docker compose restart back` ; `docker compose logs back`
Fichier de configuration modifié ou perdu	Le reprendre dans le dépôt, à la version déployée ; la configuration de l'application est aussi incluse dans l'image	`git restore docker-compose.yml`, ou `git show <sha>:docker-compose.yml`
Volume de Caddy perdu	Recréer le volume vide sans démarrer Caddy (sinon il génère une nouvelle autorité), y restaurer l'archive, puis démarrer ; sans archive, la nouvelle autorité est à importer de nouveau dans les navigateurs	`docker compose down` ; `docker compose create` ; `docker run --rm -v projet7_caddy_data:/data -v ~/sauvegardes:/sauvegarde alpine:3.24 tar xzf /sauvegarde/caddy_data-<date>.tar.gz -C /data` ; `docker compose up -d --wait`

Tableau 31 : Procédures de reprise

La sauvegarde et la restauration du volume de Caddy ont été testées : le certificat `root.crt` restauré a la même empreinte SHA-256 que l'original.

Actions automatisées qui facilitent la restauration

- `latest` **désigne toujours une version validée** : la CI ne le déplace qu'après le test de fumée. Un simple `docker compose pull` suivi de `up` remet donc en service la dernière version qui démarre, sans avoir à chercher laquelle.
- **Redéploiement automatisé d'une version stable** : après un `git revert` fusionné sur `main`, la CI reconstruit, teste et publie la version corrigée sans intervention.
- **Images conservées à l'identique** : chaque image publiée reste dans GHCR sous le SHA de son commit. Revenir à une version, c'est relancer exactement ce qui a été testé, sans rien reconstruire. Une reconstruction, elle, ne redonnerait pas une image identique : les images de base suivent leur série (Alpine 3.24, Temurin 21) et intègrent les correctifs du jour.
- **Relance automatique** des conteneurs par Docker, et healthchecks qui conditionnent le démarrage du front.

Limites

- Les données saisies sont perdues à chaque redémarrage : c'est la conception actuelle du projet, pas un oubli du plan. Une base persistante est le prérequis de toute exploitation réelle.
- L'archive du volume de Caddy n'a pas de copie hors du serveur, et la restauration n'est pas testée régulièrement : un essai périodique est prévu au § 8.3.

8. Plan de mise à jour

Le plan distingue la mise à jour courante, automatisée, et les montées de version majeure, que les métriques désignent comme la principale dette du projet : 37 % des runs de CI des pull requests de Dependabot échouent (§ 6.3), et le JAR du back embarque des failles critiques connues (§ 5.2).

8.1 Mise à jour de l'application

Mises à jour courantes : Dependabot

Chaque semaine, Dependabot propose les versions mineures et correctives, après un délai de 7 jours : une pull request groupée pour Gradle, une pour npm, une par image du Dockerfile. Ses mises à jour de sécurité n'attendent pas ce délai. Les images de la stack ELK, outil du poste de développement, se mettent à jour à la main. Chaque pull request passe la CI complète comme un changement de code. Une fois relue et fusionnée, elle est déployée automatiquement.

Une pull request rouge n'est pas fusionnée : elle signale une incompatibilité. Aujourd'hui, celles du front échouent parce qu'Angular 17 n'accepte pas TypeScript au-delà de 5.4 : seule une montée de version majeure les débloquera.

Montées de version majeure

Composant	Version actuelle	Fin du support	Cible
Spring Boot	3.2.5	Support gratuit terminé le 31/12/2024	4.1, supportée jusqu'au 31/07/2027
Angular	17.3	Terminé le 15/05/2025	22, version active
Java (images)	21	31/12/2029 (Temurin)	Inchangé
Node.js (build du front)	22	30/04/2027	Version LTS suivante avant cette date

Tableau 32 : Versions à faire évoluer (vérifiées le 05/11/2026 sur endoflife.date)

Méthode :

- **D'abord des tests** : les tests actuels ne sont que des tests de fumée (§ 4.1). Avant une montée majeure, il faut des tests du comportement de l'API et des services front (plan d'action, § 5.3), sans quoi une CI verte ne prouve pas grand-chose.
- **Une version majeure à la fois, une pull request par étape** : Angular avec `ng update`, de 17 à 18, puis 19, et ainsi de suite jusqu'à 22 ; Spring Boot d'abord vers la dernière 3.5, en corrigeant les dépréciations, puis vers la 4.
- **Chaque étape doit passer la CI et le test de fumée** ; Trivy et SonarQube mesurent le gain (failles critiques du JAR, alertes).

Images Docker

Les images de base sont fixées par version (Alpine 3.24, Temurin 21, Node 22, Caddy 2.11) et mises à jour par Dependabot. Une image vieillit sans qu'on la touche : le scan nocturne de Trivy révèle les failles publiées après coup, et une reconstruction régulière intègre les correctifs. Le binaire Caddy, recompilé pour corriger ses dépendances (§ 3.1), sera abandonné pour l'image officielle dès qu'une version de Caddy intégrera ces correctifs. L'exception de `.trivyignore.yaml` expire le 31/12/2026 et sera alors réexaminée.

8.2 Mise à jour du pipeline CI/CD

- **Actions GitHub** : fixées par SHA de commit, mises à jour par Dependabot dans une pull request groupée et hebdomadaire. Chaque mise à jour se relit avant fusion (notes de version, éditeur de l'action) : c'est du code tiers qui s'exécute avec les droits du pipeline (§ 5.2).
- **Image de Trivy** : fixée par version et par digest dans `ci.yml`, mise à jour à la main après lecture des notes de version.
- **Runners** : `ubuntu-latest` change de version d'Ubuntu quand GitHub le décide ; un échec inexpliqué après un tel changement se vérifie dans les annonces de GitHub.
- **Jetons SonarQube** : renouvelés selon la date d'expiration à leur donner (plan d'action, § 5.3).
- **Script des métriques** : il repère les déploiements par le nom des jobs (`Image`, `Test de fumée`). Renommer un job demande de mettre à jour le script dans la même pull request.

8.3 Fréquence et bonnes pratiques

Fréquence	Action
Chaque semaine	Relire et fusionner les pull requests de Dependabot ; lire les verdicts SonarQube et Trivy des derniers runs ; suivre les badges DORA du README
Chaque mois	Tester une restauration : déployer la version précédente sur un poste à partir de son SHA, puis revenir à `latest`
Chaque trimestre	Vérifier les fins de support (Spring Boot, Angular, Node.js, Java, Alpine) et planifier les montées de version ; réexaminer les exceptions de Trivy
À chaque évolution de l'architecture	Mettre à jour ces plans : nouvelle base de données, nouveau service ou nouvelle variable d'environnement changent la sauvegarde, le déploiement et la reprise

Tableau 33 : Calendrier de maintenance

Bonnes pratiques :

- toute mise à jour passe par une pull request et la CI, comme un changement de code ;
- une montée de version majeure par pull request, avec ses notes de version lues ;
- le README et cette documentation se mettent à jour dans la même pull request que le changement qu'ils décrivent ;
- une procédure de reprise qui n'a jamais été testée est considérée comme inexistante.

9. Conclusion

9.1 Améliorations apportées

MicroCRM était construit et déployé à la main, sans contrôle automatique. Sa livraison est désormais une chaîne automatisée et mesurée :

- **à chaque modification**, le back et le front sont construits, testés et analysés par SonarQube Cloud ;
- **à chaque fusion sur `main`**, les images sont construites, scannées, publiées sur GHCR, puis vérifiées par un test de fumée avant de devenir la version courante (`latest`) ;
- **chaque nuit**, la chaîne s'exécute sans publier, pour révéler ce qui change sans modification du code : une dépendance devenue vulnérable, un service externe indisponible ;
- **en continu**, Dependabot propose les mises à jour, ELK centralise les logs de l'application, et le workflow Métriques suit la performance de livraison.

Les images ont été corrigées en chemin : versions explicites, Alpine supportée, exécution sans les droits root, et plus aucune vulnérabilité haute ou critique dans les paquets système (§ 3.1).

9.2 Gains observés

Domaine	Constat	Source
Fiabilité	Aucune image n'est désignée comme version courante sans avoir démarré ; aucun échec de déploiement depuis la mise en place du déploiement continu	Test de fumée, promotion (§ 3.3) ; DORA (§ 6.2)
Rapidité	Une modification fusionnée est publiée et vérifiée en quelques minutes, sans intervention	Durée du pipeline (§ 6.3)
Qualité	La dette est mesurée et priorisée : vulnérabilités, bugs, couverture, failles des dépendances	SonarQube (§ 5.1), Trivy (§ 3.1), plan d'action (§ 5.3)
Visibilité	Les erreurs de l'application et la performance de livraison se lisent sur des tableaux de bord	Kibana (§ 6.1), badges DORA du README (§ 6.2)

Tableau 34 : Gains observés

Ces gains sont mesurés sur quelques jours d'activité et par une équipe réduite : ils fixent un point de départ, que les mêmes outils permettront de suivre dans la durée.

9.3 Recommandations pour les itérations suivantes

Par ordre de priorité, chacune reprenant une mesure de ce document :

Priorité	Recommandation	Justification
1	Écrire des tests du comportement de l'API et des services front	Les tests actuels sont des tests de fumée (§ 4.1) : sans eux, ni une CI verte ni une montée de version ne prouvent grand-chose
2	Corriger les failles simples : CORS, validation des identifiants, contrôle d'intégrité de la feuille de style Bulma chargée depuis un CDN	Vulnérabilités SonarQube et erreurs 500 vues dans ELK (§ 5.1)
3	Monter Spring Boot et Angular vers des versions supportées	Failles critiques du JAR (Trivy) et pull requests de Dependabot en échec (§ 6.3, § 8.1)
4	Ajouter l'authentification et faire de Caddy le seul point d'entrée en HTTPS	Aucun contrôle d'accès, API en HTTP (§ 5.2)
5	Passer à une base persistante, avec sa sauvegarde et ses migrations	Données perdues à chaque redémarrage (§ 7.3)

Priorité	Recommandation	Justification
6	Rendre bloquants la quality gate et le scan Trivy, et créer les premières alertes	Une fois la dette résorbée, un contrôle qui ne bloque pas finit ignoré (§ 5.2, § 6.4)

Tableau 35 : Recommandations

10. Annexes

Les annexes illustrent le fonctionnement de la chaîne. Les captures et cette documentation se trouvent aussi dans le dossier `docs/` du dépôt.

10.1 Fichiers de la chaîne CI/CD

Fichier	Rôle
`.github/workflows/ci.yml`	Pipeline CI/CD : build, tests, analyse, images, test de fumée, promotion (§ 2.1)
`.github/workflows/metriques.yml`	Calcul des métriques DORA et des KPI, badges du README (§ 6.2)
`.github/dependabot.yml`	Mises à jour automatiques des dépendances (§ 8.1)
`Dockerfile`, `docker-compose.yml`	Images du back et du front, lancement de l'application (§ 3)
`docker-compose-elk.yml`, `elk/`	Stack ELK locale et tableau de bord Kibana (§ 6.1)
`scripts/metriques-ci.mjs`	Calcul des métriques, aussi utilisable en local (§ 6.2)
`.trivyignore.yaml`	Exceptions datées du rapport Trivy (§ 5.2)

Tableau 36 : Fichiers du dépôt

10.2 Commandes utiles

Besoin	Commande
Lancer l'application à partir des sources	`docker compose up --build --wait`
Déployer les dernières images validées	`docker compose pull` puis `docker compose up -d --no-build --wait`
Déployer une version précise	`MICROCRM_TAG=<sha> docker compose pull` puis `MICROCRM_TAG=<sha> docker compose up -d --no-build --wait`
État et journaux des services	`docker compose ps` ; `docker compose logs back`
Lancer la stack ELK	`docker compose -f docker-compose-elk.yml up -d`
Générer du trafic de démonstration	`elk/generer-trafic.sh 60`
Calculer les métriques en local	`node scripts/metriques-ci.mjs --cache /tmp/metriques-ci > metriques.md`
Tests du back, du front	`cd back && ./gradlew test` ; `cd front && npm test`

Tableau 37 : Commandes utiles

10.3 Captures

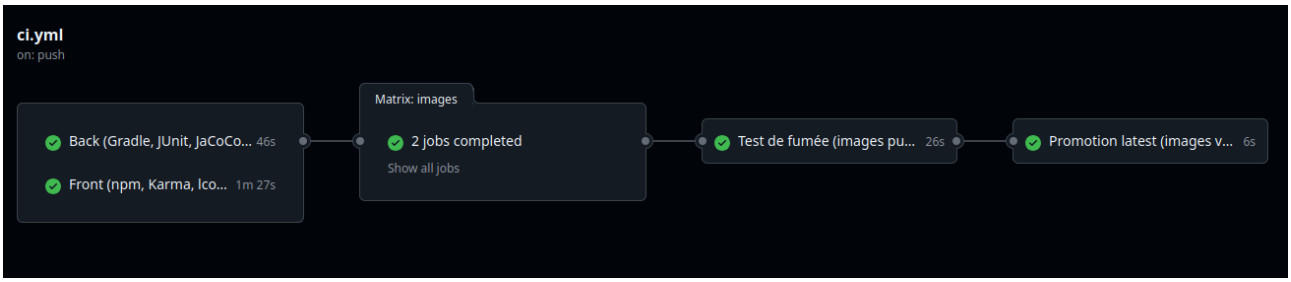


Figure 3 : Run CI sur main : enchaînement des jobs et durées

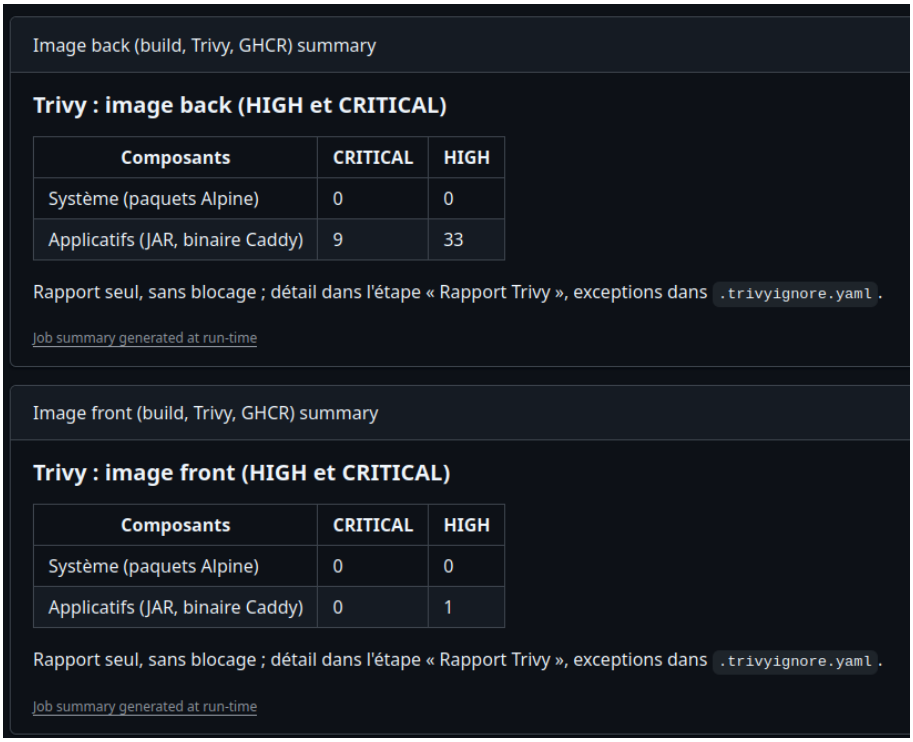


Figure 4 : Synthèse Trivy des images dans le récapitulatif du run

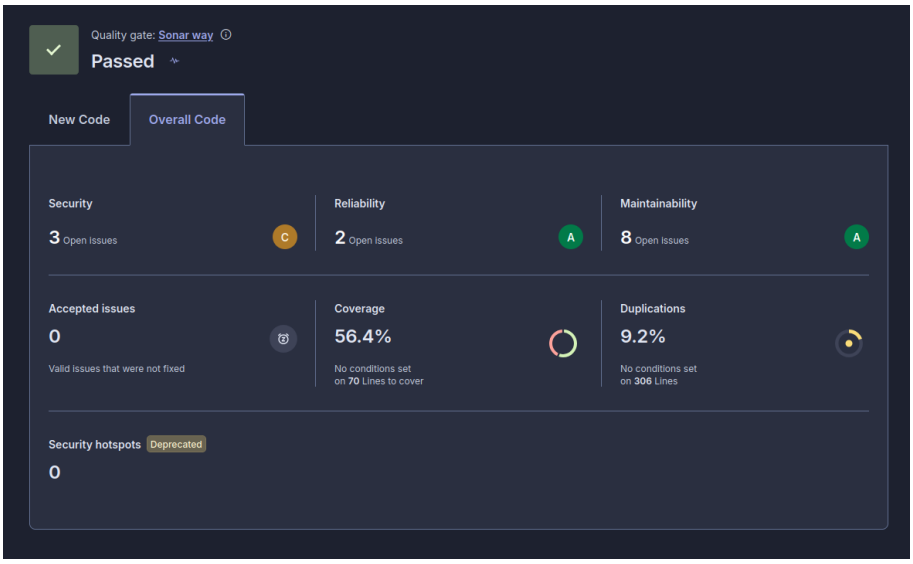


Figure 5 : SonarQube Cloud, back : quality gate réussie, dette existante visible

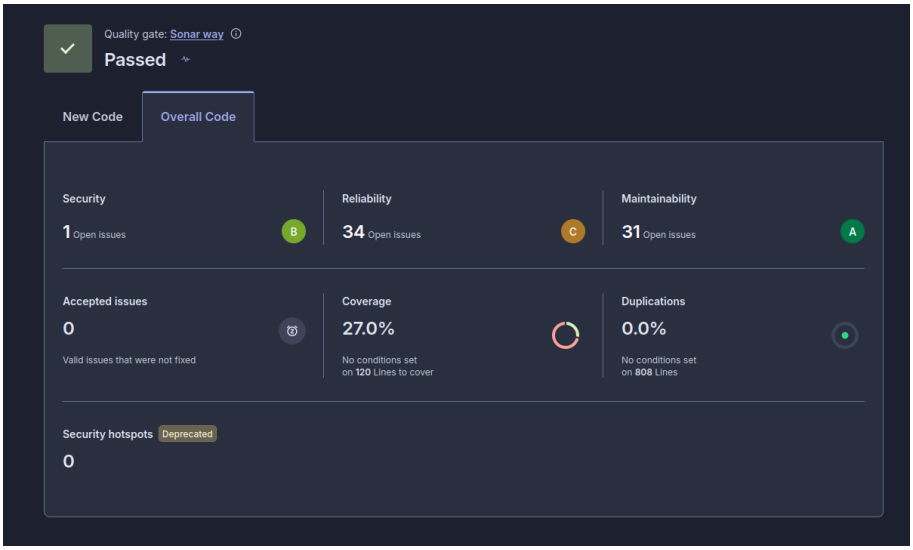
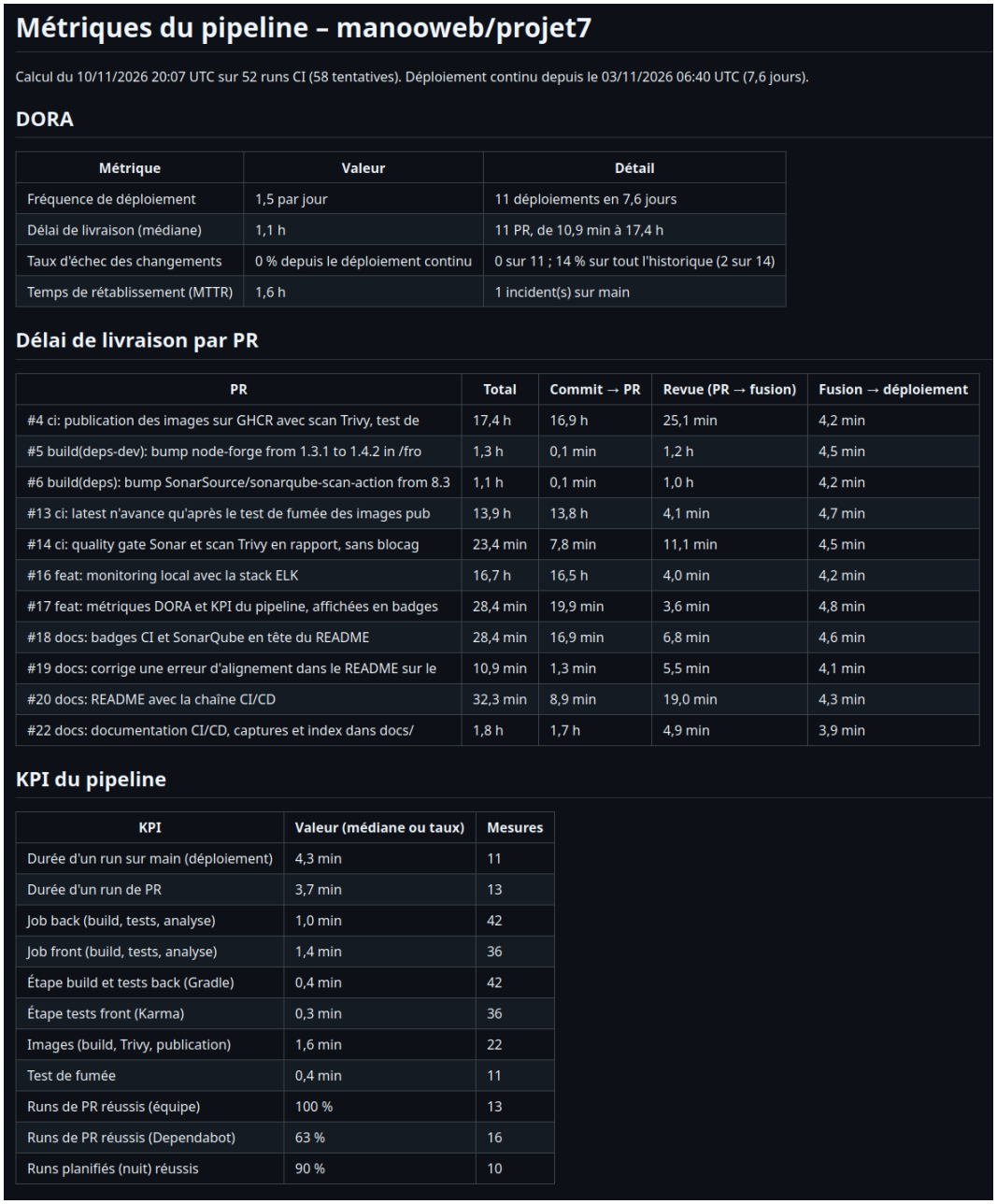


Figure 6 : SonarQube Cloud, front : quality gate réussie, dette existante visible



Figure 7 : Badges du README : CI, SonarQube et DORA



Métriques du pipeline – manooweb/projet7

Calcul du 10/11/2026 20:07 UTC sur 52 runs CI (58 tentatives). Déploiement continu depuis le 03/11/2026 06:40 UTC (7,6 jours).

DORA

Métrique	Valeur	Détail
Fréquence de déploiement	1,5 par jour	11 déploiements en 7,6 jours
Délai de livraison (médiane)	1,1 h	11 PR, de 10,9 min à 17,4 h
Taux d'échec des changements	0 % depuis le déploiement continu	0 sur 11 ; 14 % sur tout l'historique (2 sur 14)
Temps de rétablissement (MTTR)	1,6 h	1 incident(s) sur main

Délai de livraison par PR

PR	Total	Commit → PR	Revue (PR → fusion)	Fusion → déploiement
#4 ci: publication des images sur GHCR avec scan Trivy, test de	17,4 h	16,9 h	25,1 min	4,2 min
#5 build(deps-dev): bump node-forge from 1.3.1 to 1.4.2 in /fro	1,3 h	0,1 min	1,2 h	4,5 min
#6 build(deps): bump SonarSource/sonarqube-scan-action from 8.3	1,1 h	0,1 min	1,0 h	4,2 min
#13 ci: latest n'avance qu'après le test de fumée des images pub	13,9 h	13,8 h	4,1 min	4,7 min
#14 ci: quality gate Sonar et scan Trivy en rapport, sans blocag	23,4 min	7,8 min	11,1 min	4,5 min
#16 feat: monitoring local avec la stack ELK	16,7 h	16,5 h	4,0 min	4,2 min
#17 feat: métriques DORA et KPI du pipeline, affichées en badges	28,4 min	19,9 min	3,6 min	4,8 min
#18 docs: badges CI et SonarQube en tête du README	28,4 min	16,9 min	6,8 min	4,6 min
#19 docs: corrige une erreur d'alignement dans le README sur le	10,9 min	1,3 min	5,5 min	4,1 min
#20 docs: README avec la chaîne CI/CD	32,3 min	8,9 min	19,0 min	4,3 min
#22 docs: documentation CI/CD, captures et index dans docs/	1,8 h	1,7 h	4,9 min	3,9 min

KPI du pipeline

KPI	Valeur (médiane ou taux)	Mesures
Durée d'un run sur main (déploiement)	4,3 min	11
Durée d'un run de PR	3,7 min	13
Job back (build, tests, analyse)	1,0 min	42
Job front (build, tests, analyse)	1,4 min	36
Étape build et tests back (Gradle)	0,4 min	42
Étape tests front (Karma)	0,3 min	36
Images (build, Trivy, publication)	1,6 min	22
Test de fumée	0,4 min	11
Runs de PR réussis (équipe)	100 %	13
Runs de PR réussis (Dependabot)	63 %	16
Runs planifiés (nuit) réussis	90 %	10

Figure 8 : Rapport du workflow Métriques